Node.js Fundamentals

Day 3 — The Runtime, Async Model & Developer Tooling

Table of Contents

1. Blocking vs Non-Blocking Code
2. The Callback Pattern
3. The Event Loop
4. NVM — Node Version Manager
5. NPM — Node Package Manager
6. Lab 1: Setup Node Environment
7. Lab 2: Build CLI Task Manager (In-Memory)
8. Lab 3: Persist Tasks to File
9. Optional Challenge: Mark Tasks Done
10. Key Takeaways
11. Common Mistakes & Debugging Tips



Blocking vs Non-Blocking Code

The most critical concept in Node.js — single-threaded means blocking freezes everything.

Blocking (Synchronous)

```
const fs = require("fs");

console.log("Before read");

const data = fs.readFileSync("data.txt", "utf8");
// BLOCKS here until file is fully read
console.log(data);

console.log("After read");

// Output (always in this order):
// Before read
// <file contents>
// After read
```

⊗ The JS thread is stuck waiting for the OS. In a server, this freezes ALL other requests.

Non-Blocking (Asynchronous)

```
const fs = require("fs");

console.log("Before read");

fs.readFile("data.txt", "utf8", (err, data) => {
  // Called LATER, when the file is ready
  if (err) {
    console.error("Error:", err.message);
    return;
  }
  console.log(data);
});

console.log("After read"); // Runs BEFORE file is done!

// Output:
// Before read
// After read
// <file contents>
```

✓ Node delegates to libuv/OS, continues executing, and calls the callback when data is ready.

When is Sync OK?

- CLI scripts that run once and exit
- Application startup — reading config before server starts
- Build tools and one-off scripts

Never use sync APIs inside request handlers or event callbacks in a running server.

The Callback Pattern

Error-first callbacks are Node's original async convention: function(err, result)

1

Standard Pattern

```
fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Failed to read:", err.message);
    return; // Always return after handling the error!
  }
  console.log("File contents:", data);
});
```

2

Reading Files (with path.join)

```
const fs = require("fs");
const path = require("path");

// Always use path.join for safe file paths
const filePath = path.join(__dirname, "data.txt");

fs.readFile(filePath, "utf8", (err, data) => {
  if (err) {
    if (err.code === "ENOENT") {
      console.log("File does not exist yet");
    } else {
      console.error("Unexpected error:", err.message);
    }
    return;
  }
  console.log(data);
});
```

3

Writing Files

```
const fs = require("fs");

fs.writeFile("note.txt", "Hello, Node!", "utf8", (err) => {
  if (err) {
    console.error("Write failed:", err.message);
    return;
  }
  console.log("File written successfully");
});
```

4

Appending to Files

```
fs.appendFile("log.txt", "New log entry\n", "utf8", (err) => {
  if (err) console.error(err.message);
});
```

Callback Hell

```
fs.readFile("config.json", "utf8", (err, config) => {
  if (err) return console.error(err);
  fs.readFile("users.json", "utf8", (err, users) => {
    if (err) return console.error(err);
    fs.writeFile("output.json", process(config, users), (err) => {
      if (err) return console.error(err);
      console.log("Done"); // Three levels deep and growing...
    });
  });
});
```

ⓘ We solve Callback Hell with Promises and async/await on Day 3.

The Event Loop

How Node handles many concurrent operations on a single thread

01

timers
setTimeout, setInterval callbacks

02

I/O callbacks
fs, network callbacks

03

idle
internal use

04

poll
retrieve new I/O events; blocks here if empty

05

check
setImmediate callbacks

06

close callbacks
e.g. socket.on('close', ...)

ⓘ Between each phase: microtasks run (Promise .then() callbacks, process.nextTick())

Live Example — Execution Order

```
console.log("1 - Start");        // synchronous → call stack

setTimeout(() => {
  console.log("4 - Timer done");  // macrotask → timer phase
}, 0);

Promise.resolve().then(() => {
  console.log("3 - Promise");     // microtask → runs before next event loop phase
});

console.log("2 - End");          // synchronous → call stack

// Output:
// 1 - Start
// 2 - End
// 3 - Promise       ← microtasks run BEFORE timers
// 4 - Timer done
```

Microtask Priority

process.nextTick
runs at end of current operation, before I/O (highest priority)

Promise.then
microtask queue (second)

setTimeout
timer phase, next loop iteration (lowest)

```
setTimeout(() => console.log("setTimeout"), 0);
Promise.resolve().then(() => console.log("Promise.then"));
process.nextTick(() => console.log("nextTick"));
console.log("synchronous");

// Output:
// synchronous
// nextTick
// Promise.then
// setTimeout
```

ⓘ Further reading: **An Animated Guide to Node.js Event Loop — dev.to**

NVM — Node Version Manager

Install and switch between multiple Node.js versions — essential for multi-project workflows

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Restart your terminal (or source ~/.zshrc / source ~/.bashrc), then verify with: nvm --version

Common NVM Commands

```
nvm install --lts
```
Install the latest LTS version (recommended)

```
nvm install 20
```
Install a specific major version

```
nvm use 20
```
Switch to Node 20 in current shell

```
nvm alias default 20
```
Set Node 20 as default for new shells

```
nvm ls
```
List all installed versions

```
nvm ls-remote
```
List all available versions to install

```
nvm current
```
Show currently active version

Why Use NVM?



Different projects may require different Node versions



Avoids polluting global system Node installation



Makes version upgrades easy and reversible



Aligns your dev environment with production constraints

Windows users: Use nvm-windows instead — it's a separate tool with a very similar interface.

NPM — Node Package Manager

Handles dependencies, scripts, and project metadata — built into Node.js

Initialize a Project

```
mkdir task-cli
cd task-cli
npm init -y     # -y accepts all defaults
```

Show the resulting package.json:

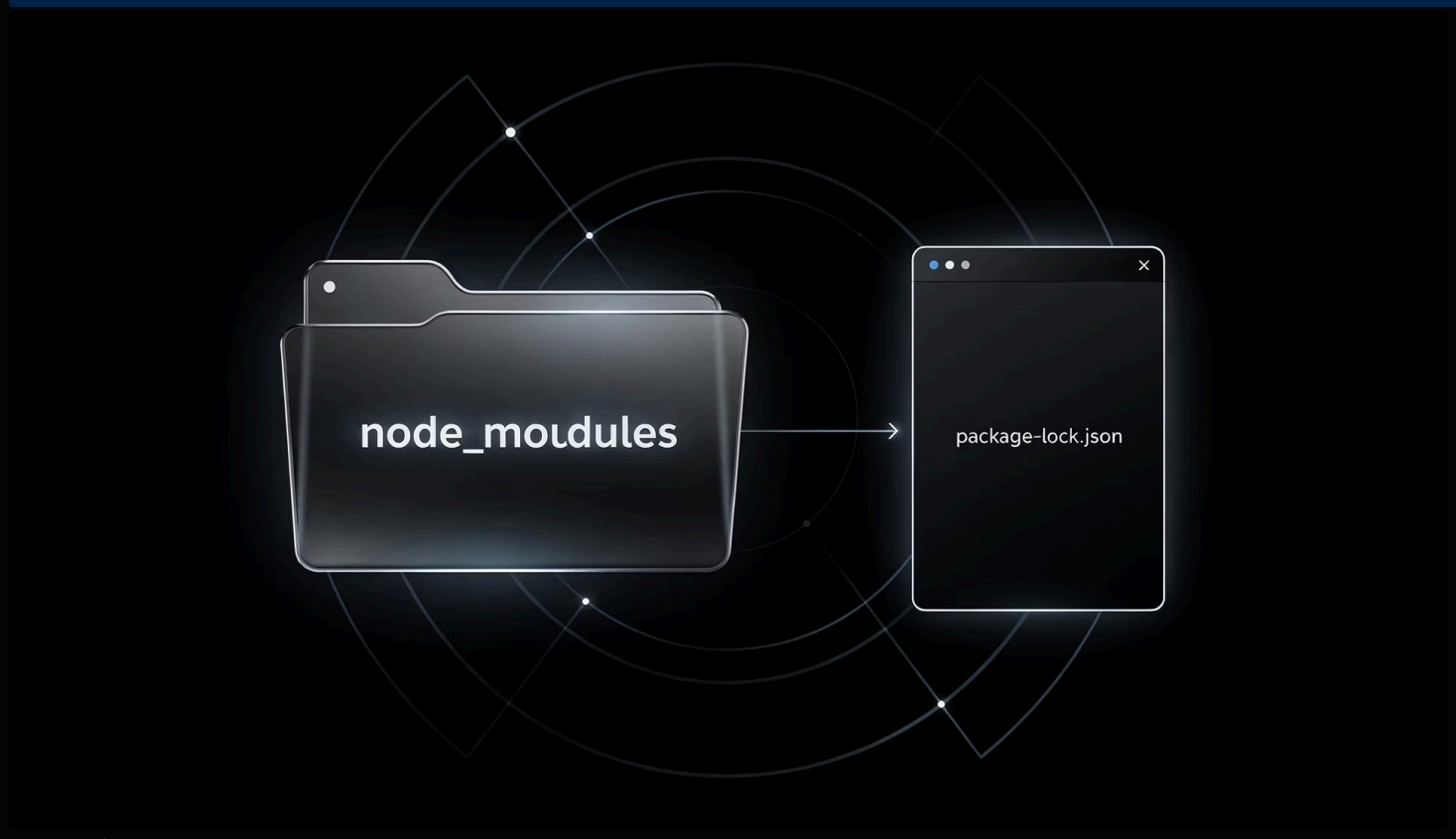
```
{
  "name": "task-cli",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo "Error: no test specified" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Install Packages

```
npm install chalk            # production dependency
npm install nodemon --save-dev # development-only dependency
npm install                  # install all deps listed in package.json
```

node_modules/ — the actual package code (never commit this to git)

package-lock.json — exact version lockfile (do commit this)



NPM Scripts

— show the scripts section:

```
{
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "node test.js"
  }
}
```

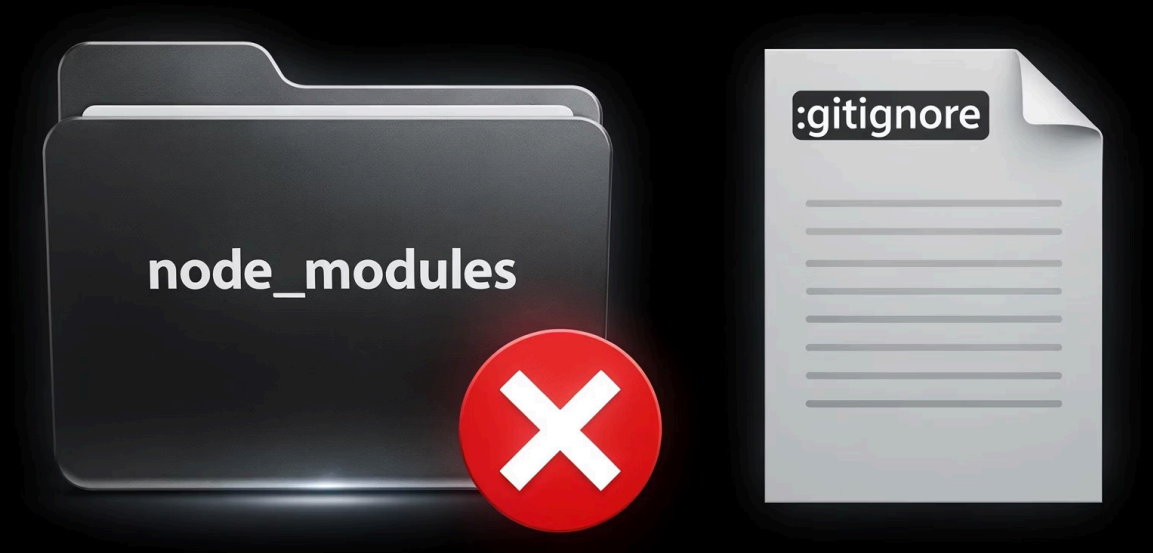
Run commands:

```
npm run start
npm run dev
npm test        # "test" and "start" don't need the "run" keyword
```

- Standardised entry points for any project
- Abstracts the actual command behind a memorable name
- Can chain multiple commands with && or use tools like npm-run-all

.gitignore — Don't Commit node_modules

```
node_modules/
tasks.json
.env
```



Lab 1: Setup Node Environment

Get your development environment ready for the rest of the course

01

Install NVM

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.zshrc   # or ~/.bashrc
nvm --version
```

02

Install Node LTS

```
nvm install --lts
nvm use --lts
node --version    # should print v20.x.x or similar
npm --version
```

03

Create Your Project

```
mkdir task-cli
cd task-cli
npm init -y
```

04

Create index.js

```
console.log("Hello from Node!", process.version);
console.log("Args:", process.argv.slice(2));
```

05

Run It

```
node index.js
node index.js hello world
```

06

Add a Start Script

In package.json, add:

```
"scripts": {
  "start": "node index.js"
}
```

Then run: `npm start`

You're ready! Your environment is set up and you've run your first Node.js script.

Key Takeaways

Day 1 — Node.js Fundamentals and the Runtime

Single-Threaded, Non-Blocking — Node runs on one thread. Blocking operations freeze everything. Always prefer async APIs in servers.

Callbacks Are Error-First — The convention is (err, result). Always check err before using result, and always return after handling an error.

The Event Loop Is Your Friend — Understanding phases (timers → I/O → poll → check) and microtask priority (nextTick > Promise.then > setTimeout) is essential for debugging async code.

NVM for Version Management — Use nvm to install, switch, and pin Node versions per project. Never rely on a global system Node.

NPM Is More Than Install — Use npm scripts to standardise commands, package.json to document your project, and .gitignore to keep node_modules out of git.

Bridge to Day 2

You've built a local CLI tool. Tomorrow, we expose it to the outside world by turning it into an HTTP server — and learn how the web actually works.

Common Mistakes & Debugging Tips

Avoid these pitfalls and use these tools to debug faster

Common Mistakes

- **Using readFileSync inside a request handler** — freezes the server for all users
- **Forgetting to return after handling an error in a callback** — code continues executing
- **Not handling the ENOENT error code** — crashes on first run before file exists
- **Committing node_modules to git** — bloats your repo with thousands of files
- **Using process.argv[0] or [1]** — those are node and the script path; your args start at [2]

Quick Debugging Reference

Problem	Debug Command
What are my CLI args?	console.log(process.argv)
What's in my JSON file?	cat tasks.json
Is my file being found?	console.log(TASKS_FILE)
Why is my callback not running?	Check if you're calling the async function at all
Unexpected undefined?	Log the full object before accessing properties

End of Day 3 — Node.js Fundamentals and the Runtime